PCG Arena: A Platform for Blind A/B Testing of Procedural Content Generators with Direct Preference Optimization

(PCG Arena: Platforma do Ślepego Testowania A/B
Generatorów Treści Proceduralnych
z Optymalizacją Preferencji)

Antoni Czapski

Praca magisterska

Promotor: dr Jakub Kowalski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

March 2026

Abstract

Evaluating procedural content generators (PCGs) for video games poses significant methodological challenges, as player enjoyment is inherently subjective and difficult to quantify through automated metrics alone. This thesis presents PCG Arena, a web-based platform designed for blind A/B testing of Super Mario Bros level generators through human preference data. The platform enables players to play two procedurally generated levels side-by-side in their browser, vote for their preferred level, and optionally tag gameplay characteristics. Votes are aggregated using the Glicko-2 rating system, providing uncertainty-aware rankings with confidence intervals.

Building on the collected data, we conduct an extensive exploratory data analysis investigating what makes levels good, whether player preferences are consistent, and whether subjective tags correspond to measurable gameplay differences. These findings inform the development of a heuristic Judge Function achieving $r = 0.736$ correlation with human preferences. Finally, we present MarioDPO, a level generator fine-tuned with Direct Preference Optimization that achieves the highest win rate (83.3%) among all PCG methods tested, second only to hand-crafted original Nintendo levels.

Ewaluacja generatorów treści proceduralnych (PCG) dla gier wideo stanowi istotne wyzwanie metodologiczne, ponieważ przyjemność gracza jest z natury subiektywna i trudna do zmierzenia za pomocą automatycznych metryk. Niniejsza praca przedstawia PCG Arenę — platformę internetową do ślepego testowania A/B generatorów poziomów do gry Super Mario Bros poprzez dane o preferencjach graczy. Platforma umożliwia graczom rozgrywanie dwóch proceduralnie generowanych poziomów obok siebie w przeglądarce, głosowanie na preferowany poziom oraz opcjonalne tagowanie cech rozgrywki. Głosy są agregowane za pomocą systemu ratingowego Glicko-2, który zapewnia rankingi z przedziałami ufności.

Na podstawie zebranych danych przeprowadzono rozległą analizę eksploracyjną badającą, co czyni poziomy dobrymi, czy preferencje graczy są spójne oraz czy subiektywne tagi odpowiadają mierzalnym różnicom w rozgrywce. Wyniki te posłużyły do opracowania funkcji oceny (Judge Function), która koreluje z preferencjami ludzi na poziomie $r = 0.736$. Finalnie, praca przedstawia MarioDPO — generator poziomów wytrenowany metodą Direct Preference Optimization, który osiąga najwyższy wskaźnik wygranych (83.3%) spośród wszystkich metod PCG, ustępując jedynie oryginalnym poziomom Nintendo.

Contents

Chapter 1

Introduction

1.1 Procedural Content Generation in Video Games

Procedural Content Generation (PCG) refers to the algorithmic creation of game content—levels, maps, characters, items, narratives—with limited or indirect human input [29, 28]. PCG has a long commercial history: *Rogue* (1980) and *Elite* (1984) used algorithmic generation to overcome memory constraints, and modern titles such as *Minecraft*, *No Man's Sky*, and *Spelunky* rely on procedural generation as a core design pillar. Academic interest in PCG has grown steadily since the mid-2000s, producing a rich landscape of generation techniques spanning constructive rules, evolutionary search, formal grammars, neural networks, and reinforcement learning.

Among the many game domains studied, Super Mario Bros (SMB) has emerged as the *de facto* benchmark for PCG research. Its 2D tile-grid format provides a rich yet tractable design space, its physics are well understood, and the open-source Mario AI Framework [9] offers a standardised environment for both automated and human evaluation. Over 15 years of research has produced dozens of Mario level generators spanning every major PCG paradigm—yet evaluating and comparing them remains an open challenge.

1.2 The Evaluation Problem

Evaluating procedural content generators is fundamentally difficult because the ultimate quality criterion—player enjoyment—is subjective, multidimensional, and expensive to measure. The PCG literature has developed three families of evaluation methods, each with significant limitations.

Automated structural metrics, such as linearity, leniency, and density, can be computed directly from level tilemaps and are widely used for characterising generator output [22, 8]. However, Mariño et al. [11] demonstrated that these metrics

only weakly correlate with human-rated difficulty and enjoyment, concluding that "current computational metrics should not be used in lieu of user studies." Multiple incompatible definitions of the same metric—at least three formulations of leniency exist in the literature—further complicate cross-study comparisons.

Agent-based testing uses AI controllers—typically the A* agent from the 2009 Mario AI Championship—as player proxies. Agent-derived features such as jump count or completion fraction provide scalable difficulty estimates [30], but superhuman agents can solve levels that human players cannot. Difficulty estimates depend on the specific agent policy, introducing a systematic bias that is rarely quantified.

Human evaluation studies remain the gold standard but are expensive and typically small-scale: controlled lab studies use 15–37 participants [19, 11], while larger web-based studies sacrifice experimental control [15]. Critically, no prior platform offers *blind* comparison—when participants know which algorithm produced a level, expectations contaminate their judgments.

This persistent gap between scalable-but-inaccurate automated metrics and valid-but-expensive human studies motivates the work presented in this thesis.

1.3 Research Questions and Contributions

This thesis addresses the PCG evaluation gap through the design, deployment, and analysis of *PCG Arena*, a web-based platform for blind A/B testing of Mario level generators. We investigate three research questions:

RQ1: What makes a good level?
Do structural features of levels—difficulty, variety, path freedom, hazard density—predict human preferences? Do commonly used automated metrics align with player judgments?

RQ2: Are player preferences consistent?
Do different players agree on which levels are better, or do preferences fragment into identifiable clusters?

RQ3: Do subjective tags have measurable correlates?
Do qualitative labels assigned by players (e.g., "fun," "too hard," "creative") correspond to measurable differences in gameplay telemetry and level structure?

The thesis makes four contributions:

1. **PCG Arena**, a persistent, open-access web platform for blind A/B testing of

Super Mario Bros level generators, featuring browser-based gameplay, Glicko-2 uncertainty-aware rankings, and a novel matchmaking algorithm (AGIS).

2. **A user study** collecting 571 pairwise preference votes from 27 players across 15 generators (748 levels), with gameplay trajectories and subjective tags—one of the largest blind human evaluation datasets in Mario PCG research.

3. **A Judge Function**, a heuristic preference predictor achieving $r = 0.736$ correlation with human preferences, providing a scalable proxy for human evaluation.

4. **MarioDPO**, a level generator fine-tuned with Direct Preference Optimisation on arena votes, achieving the highest win rate (83.3%) among all PCG methods tested, second only to hand-crafted Nintendo originals.

1.4 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 surveys background and related work on procedural content generation, evaluation methodology, rating systems, and preference learning. Chapter 3 describes the design and implementation of the PCG Arena platform. Chapter 4 presents the user study and an extensive exploratory data analysis addressing the three research questions. Chapter 5 details the development of the Judge Function and the MarioDPO generator. Chapter 6 summarises contributions, discusses limitations, and outlines future directions.

Chapter 2

Background and Related Work

2.1 Procedural Content Generation

Procedural Content Generation (PCG) is the algorithmic creation of game content with limited or indirect human input [29]. The most common taxonomy [29, 28] distinguishes five generation paradigms:

Constructive / Rule-based
Sequentially place content according to hand-authored rules or grammars. Fast and reliable, but limited in expressiveness.

Search-based (SBPCG)
Frame generation as optimisation over a fitness function, typically using evolutionary algorithms. Flexible but computationally expensive.

Grammar / Pattern-based
Use formal grammars, design patterns, or n-grams to encode structural idioms. Captures human design style but requires pattern libraries.

Machine Learning (PCGML)
Train neural networks on existing human-designed content. Data-driven and expressive, but may lack diversity or playability guarantees.

Reinforcement Learning (PCGRL)
Frame level design as a Markov Decision Process; train an agent to construct content. Unlike PCGML, requires no corpus of existing levels—only a reward function that encodes the desired content properties.

These paradigms are not mutually exclusive; many modern systems are hybrids. For instance, MarioGAN [30] combines a learned GAN with CMA-ES search in its latent space. A further useful distinction is between *offline* generation (content precomputed before the game ships) and *online* generation (content created at runtime,

potentially adapting to the player). Online generation enables personalisation but imposes stricter time budgets—a theme central to experience-driven PCG [31]. A desirable property of any generator is **controllability**: the ability to accept parameters describing desired content features and produce content that complies.

2.1.1 Search-Based Approaches

Search-Based PCG (SBPCG) frames content generation as an optimisation problem [29]. A *fitness function* encodes the desired properties of the content (e.g., playability, target difficulty, aesthetic criteria), and an evolutionary algorithm—typically a genetic algorithm or CMA-ES—searches a space of candidate solutions to maximise it.

The approach is attractive because fitness functions provide explicit, modular designer control: one can compose playability constraints, difficulty targets, and diversity objectives into a single optimisation criterion. However, SBPCG has two well-known limitations. First, each fitness evaluation may require an expensive simulation (e.g., running an A* agent through a candidate level). Second, designing good fitness functions is itself a difficult problem—one that mirrors the evaluation challenge motivating this thesis.

The Mario AI Level Generation Competition [19] featured several search-based entries. More recent work has applied CMA-ES to the latent space of trained generative models [30], effectively combining search-based and ML-based paradigms.

2.1.2 Grammar-Based and Rule-Based Approaches

Constructive generators build content sequentially according to hand-authored rules, typically working left-to-right through a level. The Notch generator shipped with *Infinite Mario Bros* [9] is the canonical example: it iterates through section templates (straight, hill, tubes, gaps, cannons) with fixed selection probabilities and basic playability checks. Constructive methods are fast and guarantee structural validity, but their expressive range is narrow.

Grammar-based approaches extend this idea by encoding design rules in a formal grammar. Shaker et al. [21] used **Grammatical Evolution** (GE) to map integer genotypes to level phenotypes via context-free grammar productions, combining the interpretability of grammars with evolutionary search. Smith and Whitehead [22] introduced **Launchpad**, a rhythm-based generator that maps player actions to level geometry via a design grammar. The same paper introduced **Expressive Range Analysis** (ERA)—the practice of generating a large sample of levels, computing structural metrics, and visualising their joint distribution—which has become the dominant method for characterising PCG generators.

2.1.3 Machine Learning-Based Generation (PCGML)

Machine learning approaches to PCG (PCGML) train neural networks on existing content to learn implicit design knowledge [28]. Several architectures have been applied to Mario level generation:

Sequence models. Summerville and Mateas [26] trained a 3-layer LSTM to generate levels column-by-column using a "snaking" tokenisation that preserves vertical adjacency. Co-generating levels with A*-agent play traces achieved 97% playability.

Generative adversarial networks. MarioGAN [30] trains a Deep Convolutional GAN on sliding-window segments from a single original SMB level. CMA-ES then searches the GAN's latent space to optimise fitness functions based on tile distributions or agent-derived playability. This work established the **Latent Variable Evolution** (LVE) paradigm.

Large language models. MarioGPT [25] fine-tunes DistilGPT-2 on column-tokenised Mario levels with text-prompt conditioning (e.g., "many pipes, many enemies, low elevation"), enabling the first natural-language-controllable level generator. 88% of generated levels are playable without post-processing.

Diffusion models. MarioDiffusion [18] uses a text-conditioned UNet denoiser to generate 16×16-tile scenes from natural-language captions, representing the latest generation of multimodal PCG.

Reinforcement learning. PCGRL [10] formulates level design as an MDP and trains an RL agent to edit tile grids, bypassing the need for training data entirely.

2.2 Super Mario Bros as a PCG Benchmark

Super Mario Bros has become the *de facto* benchmark domain for PCG research. Its 2D tile-grid format provides a rich yet tractable design space, its physics introduce no new mechanics after the first level (isolating level design as the sole variable of interest), nearly all experimental subjects are familiar with the game's controls and goals, and the open-source Mario AI Framework provides a standardised evaluation environment.

2.2.1 The Mario AI Framework

The Mario AI Framework [9] is an open-source Java implementation based on *Infinite Mario Bros*, a public-domain clone of Nintendo's Super Mario Bros created by Markus Persson in 2008. It served as the basis for three tracks of the Mario AI Championship (2009–2012):

1. **Gameplay Track**—build a controller that plays levels as well as possible.
2. **Learning Track**—build a controller that learns to play.
3. **Level Generation Track**—build software that generates levels for human players.

The Level Generation Track [19] was the first academic PCG competition. Generators received gameplay metrics from a test level and had 60 seconds to produce a personalised level. Six entries competed; evaluation used pairwise forced-choice ("which was more fun?") with 15 human judges. This competition established several norms that influenced subsequent research: pairwise comparison as the preferred human evaluation protocol, player telemetry as input to adaptive generators, and the A* agent as a *de facto* playability oracle.

Levels are stored as 2D character grids (ASCII tilemaps), where each character encodes a functional tile type. The Video Game Level Corpus (VGLC) [26] standardised this encoding across all 32 original SMB levels and has been widely used as training data for neural generators.

2.2.2 Mario Level Generators in the Literature

The PCG Arena platform includes 15 generators spanning all major paradigm families, selected to benchmark a broad cross-section of established PCG approaches. Table 2.1 provides a comparative overview.

Human-authored baseline. The 15 original SMB levels from Nintendo (1985), transcribed into the Mario AI Framework's format. These serve as the quality ceiling against which all algorithmic generators are measured.

Constructive generators. Four generators build levels left-to-right using probabilistic placement: the *Notch* generator shipped with Infinite Mario Bros, its *Parameterized* variant (NotchParam) exposing six tunable difficulty/style knobs [19], a *Randomized* variant (NotchParamRand) that randomly samples parameters per level, and *Hopper*, which alternates generated and pre-designed segments with adaptive probability adjustment.

Chunk-based assembly. *Occupancy-Regulated Extension* (ORE) [12] builds levels by iteratively attaching hand-authored chunks at compatible anchor points, producing structurally distinctive levels.

Search-based generators. The *Grammatical Evolution* (GE) generator [21] evolves level-construction programs via a context-free grammar. Three *Pattern-based* generators [4] represent levels as sequences of micro-patterns extracted from original SMB levels and use evolutionary search with different fitness functions: pattern count (maximising motif frequency), pattern occurrence (maximising motif diversity), and weighted pattern count (rewarding rare motifs).

Neural generators. *MarioGAN* [30] combines a DCGAN with CMA-ES latent-space search. *MarioGPT* [25] uses a fine-tuned DistilGPT-2 with text-prompt control. *MarioDiffusion* [18] generates tile scenes from natural-language captions via a diffusion model. Finally, *MarioDPO*—this thesis's contribution—fine-tunes a GPT-2-based generator with Direct Preference Optimisation using PCG Arena votes.

Generator	Paradigm	Key technique	Control	Year
Original SMB1	Human	Manual design	—	1985
Notch	Constructive	Probabilistic L-to-R	None	2008
NotchParam	Constructive	Probabilistic + 6 params	Param. knobs	2011
NotchParamRand	Constructive	Probabilistic + random	Random	2011
Hopper	Constructive	Adaptive probabilistic	Adaptive	2010
ORE	Chunk-based	Anchor-point assembly	None	2010
GE	Search	Grammatical evolution	Via grammar	2012
Pattern Count	Search	Evolutionary (pattern freq.)	Fitness wts.	2014
Pattern Occurrence	Search	Evolutionary (unique patt.)	Fitness wts.	2014
Pattern Wt. Count	Search	Evolutionary (rarity-wt.)	Fitness wts.	2014
MarioGAN	ML (GAN)	DCGAN + CMA-ES latent	Fitness fn.	2018
MarioGPT	ML (LLM)	Fine-tuned DistilGPT-2	Text prompts	2023
MarioDiffusion	ML (Diff.)	Text-conditioned UNet	Text captions	2025
MarioDPO	ML + DPO	GPT-2 + pref. alignment	Text + DPO	2026

Table 2.1: Generators included in PCG Arena. All except MarioDPO (this thesis) are previously published or reference implementations.

By including generators from all paradigm families and spanning nearly four decades of game content creation, PCG Arena tests whether human preference data can discriminate generator quality more reliably than automated metrics.

2.3 Evaluation Methods for Procedural Content

Evaluation is the central methodological challenge in PCG research and the primary motivation for PCG Arena. This section surveys the main families of evaluation methods used in the Mario PCG literature.

2.3.1 Automated Metrics

The most widely used evaluation approach computes *static structural metrics* directly from the tile grid, without simulating gameplay.

Linearity measures how closely a level's vertical profile follows a straight line. Smith and Whitehead [22] defined it via linear regression on platform midpoints; later works [8, 27] use R^2 goodness-of-fit instead, inverting the scale. These incompatible definitions complicate cross-paper comparisons.

Leniency approximates difficulty through weighted sums of game objects. At least three incompatible formulations exist in the literature, using different tile weights and normalisation schemes [22, 21, 11]. Critically, Mariño et al. [11] showed that leniency only weakly correlates with human-rated difficulty.

Compression distance (gzip-based Normalised Compression Distance) quantifies structural similarity between level pairs and is used as a diversity proxy [21, 8].

Expressive Range Analysis (ERA), introduced by Smith and Whitehead [22], generates a large sample of levels, computes two metrics per level (classically linearity vs. leniency), and visualises the joint distribution as a 2D histogram, revealing a generator's output peaks, holes, and biases. ERA has been widely adopted [8, 21] but is purely descriptive—it characterises *what* a generator produces, not whether players enjoy it.

2.3.2 Agent-Based Testing

AI agents serving as player proxies enable scalable, repeatable assessment of playability and difficulty.

Playability gates. A level is deemed playable if a strong A* agent can complete it. This binary check is the most common first stage in PCG evaluation pipelines [30, 26, 10].

Difficulty proxies. Agent-derived metrics include jump count (the number of jumps the A* agent performs, interpreted as difficulty), completion fraction (progress along the x-axis if the level is not fully completable), and virtual damage from stochastic "human-like" agents that model input timing inaccuracies [13].

Agent-based evaluation is fast, reproducible, and incurs no participant cost, but agent bias is a fundamental concern: superhuman agents can solve levels humans cannot, and difficulty estimates depend on the specific agent policy. These limitations are rarely quantified in the literature.

2.3.3 Human Evaluation Studies

Human studies remain the gold standard for PCG evaluation but vary widely in scale and methodology.

Pairwise forced-choice. The Mario AI Championship [19] asked 15 judges to play two levels and select "which was more fun?" without further defining "fun." This protocol directly inspired PCG Arena's design.

Likert-scale ratings. Mariño et al. [11] used 7-point Likert scales for enjoyment, aesthetics, and difficulty ($n = 37$). Summerville et al. [27] collected ratings across 85 design metrics and showed that inter-rater agreement imposes ceilings on

achievable metric–human correlations.

Large-scale web studies. Pedersen et al. [15] recruited 181 subjects via the web to play *Infinite Mario Bros* variants and report pairwise affective preferences, although only the first 120 participants (240 game pairs) were used in the analysis. A follow-up by Shaker et al. [20] collected 600 game-pair comparisons via an uncontrolled web applet, but the number of unique participants is not reported. Both studies date from 2010–2011 and predate the wave of ML-based generators (GANs, LSTMs, diffusion models) that have since reshaped the field.

These studies collectively demonstrate that while human evaluation provides the most valid quality signal, it is expensive, typically small-scale ($n < 40$ in controlled settings), and—critically—never blind to generator identity. Moreover, even the largest prior web study was conducted before modern generative methods existed, leaving a significant gap in comparative human evaluation of contemporary Mario level generators.

2.3.4 Comparison of Approaches and Their Limitations

Table 2.2 summarises the evaluation methods employed across key papers in the Mario PCG literature.

Paper	Year	S	A	ERA	H	ML
Smith & Whitehead	2010	✓		✓		
Pedersen et al.	2010	✓			✓	✓
Shaker et al. (Champ.)	2011	✓	✓		✓	
Shaker et al. (GE)	2012	✓		✓		
Horn et al.	2014	✓		✓		
Marino et al.	2015	✓			✓	
Summerville & Mateas	2016	✓	✓			✓
Summerville et al.	2017	✓	✓		✓	✓
Volz et al. (MarioGAN)	2018	✓	✓			
Nam et al.	2024	✓	✓		✓	
PCG Arena (ours)	**2026**	✓	✓		✓	✓

Table 2.2: Evaluation methods used in Mario PCG research. S = structural metrics; A = agent-based; ERA = Expressive Range Analysis; H = human study; ML = learned predictor.

Three persistent problems emerge from this landscape:

Metrics–experience misalignment. Mariño et al. [11] showed that two generators rated identically by all computational metrics were rated *significantly differently* by human players on enjoyment. Summerville et al. [27] further demonstrated that inter-rater agreement imposes ceilings on achievable metric–human correlations,

with aesthetics being particularly noisy. Automated metrics are useful characterisation tools but unreliable quality proxies.

Small-n and non-blind studies. Most human studies use 15–37 participants in controlled settings. No prior platform offers both scale and blind comparison—when players know which algorithm produced a level, expectations contaminate judgments.

Absence of a community-standard reward model. Search-based and RL-based generators require fitness or reward functions, yet no validated reward model for Mario level quality exists. The training data needed to build such a model—large-scale human preferences over diverse generators—has been unavailable.

PCG Arena addresses these gaps by providing a persistent, blind, web-based platform for human evaluation, producing the preference data needed to (a) rank generators with statistical confidence, (b) validate automated metrics, and (c) train preference-aligned generators.

2.4 Rating Systems for Pairwise Comparisons

PCG Arena uses pairwise comparisons as its primary data collection protocol. This section reviews the mathematical foundations underlying the platform's rating engine.

2.4.1 The Elo Rating System

All modern skill-rating systems rest on the **Bradley-Terry model** [1]: each item i is assigned a strength parameter π_i, and the probability that item i is preferred to item j is

$$P(i \succ j) = \frac{\pi_i}{\pi_i + \pi_j}. \tag{2.1}$$

Elo [5] operationalised this model for competitive chess. Each player carries a scalar rating R; the expected score of player A against player B is

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}. \tag{2.2}$$

After an observed outcome $S_A \in \{0, 0.5, 1\}$, the rating updates as $R'_A = R_A + K(S_A - E_A)$, where K is a fixed gain constant. Elo's simplicity made it the standard in chess, tennis, and early online games, but its constant K cannot distinguish a well-established rating from a newcomer's, motivating Bayesian extensions.

2.4.2 The Glicko-2 Rating System

Glickman [6] recast Elo in a Bayesian framework (**Glicko**), modelling each player's

rating as a Gaussian $\mathcal{N}(\mu, \sigma^2)$ where σ—the *rating deviation* (RD)—quantifies uncertainty. Between rating periods, RD grows to reflect increased uncertainty from inactivity.

Glicko-2 [7] extends Glicko with a third parameter, *volatility* σ_v, which captures how erratic an entity's performance is. The update pipeline converts ratings to an internal scale, estimates variance and improvement from outcomes, iteratively updates volatility, and then updates RD and rating:

$$\phi' = \frac{1}{\sqrt{1/\phi^{*2} + 1/v}}, \quad \text{where } \phi^* = \sqrt{\phi^2 + \sigma_v'^2}, \tag{2.3}$$

$$\mu' \leftarrow \mu + \phi'^2 \sum_j g(\phi_j)(s_j - E_j), \tag{2.4}$$

where $g(\phi) = 1/\sqrt{1 + 3\phi^2/\pi^2}$ down-weights predictions involving uncertain opponents, s_j is the observed outcome, and E_j the expected outcome.

PCG Arena uses Glicko-2 with generators and levels treated as "players": each human vote corresponds to a match outcome. RD provides a built-in measure of rating reliability, and volatility accommodates generators whose perceived quality shifts as new levels are added. Implementation details and parameter choices are described in Section 3.6.

2.4.3 Arena-Style Evaluation Platforms

The **arena paradigm**—collecting anonymous pairwise votes from a crowd to rank competing systems—has recently been adopted beyond games.

Chatbot Arena. Chiang et al. [2] deployed a web platform where users chat simultaneously with two anonymous LLMs and vote for the better response. Over 240,000 votes from 90,000+ users have been collected; rankings are computed via Bradley-Terry MLE. An active sampling strategy concentrates votes on close model pairs, mirroring the information-gain logic that motivates PCG Arena's AGIS algorithm. Chatbot Arena is the closest structural analogue to PCG Arena outside the games domain.

LLM-as-a-Judge. Zheng et al. [32] studied using GPT-4 as a surrogate judge for pairwise evaluation, achieving >80% agreement with human preferences. PCG Arena's *Judge Function* plays an analogous role: a lightweight model predicting which of two Mario levels a human would prefer.

Sarkar and Cooper [17] applied Glicko-2 to the human computation game *Paradox*, treating puzzle-tasks as rated entities alongside players. This is the most direct precedent for PCG Arena's use of Glicko-2 to rate game content rather than human players.

Snow et al. [23] showed that aggregating labels from multiple non-expert annotators can match expert-level quality, underpinning the design assumption that a

sufficient number of non-expert pairwise votes, properly aggregated, yields a trustworthy ranking.

2.5 Preference Learning and Alignment

PCG Arena's preference data enables not only passive ranking but also active generator improvement via preference learning. This section reviews the two key paradigms: Reinforcement Learning from Human Feedback (RLHF) and its simplified successor, Direct Preference Optimisation (DPO).

2.5.1 Reinforcement Learning from Human Feedback

RLHF trains a policy to maximise a reward signal derived from human preferences rather than a hand-designed reward function. The pipeline crystallised over a sequence of four landmark papers:

1. Christiano et al. [3] proposed learning a reward model $\hat{r}$ from pairwise trajectory comparisons and optimising the policy against $\hat{r}$ via PPO. Applied to Atari and MuJoCo.
2. Ziegler et al. [33] adapted the scheme to language models (GPT-2) and introduced a KL penalty $\beta\,\mathrm{KL}[\pi\|\pi_{\mathrm{ref}}]$ to prevent drift from the pretrained model.
3. Stiennon et al. [24] scaled to summarisation (6.7B model, 64K comparisons) and showed that the learned reward predicts human preferences better than ROUGE.
4. Ouyang et al. [14] codified the three-step recipe (SFT $\rightarrow$ Reward Model $\rightarrow$ PPO). A 1.3B InstructGPT was preferred over 175B GPT-3, demonstrating that alignment quality matters more than model scale.

The RLHF pipeline addresses the same fundamental problem as PCG Arena: *metric mismatch*. Just as ROUGE fails to capture summary quality [24], automated PCG metrics fail to capture fun, aesthetics, or challenge appropriateness [11]. PCG Arena's paired-comparison voting is the game-design analogue of RLHF's preference collection phase.

2.5.2 Direct Preference Optimization

Rafailov et al. [16] observed that the optimal RLHF policy can be expressed in closed form as a function of the reward model. Substituting this solution back into the reward-model loss yields **Direct Preference Optimisation** (DPO), which directly fine-tunes the policy using preference pairs—eliminating both the reward model and the RL loop:

$$\mathcal{L}_{\mathrm{DPO}}(\pi_\theta;\pi_{\mathrm{ref}}) = -\mathbb{E}_{(x,\,y_w,\,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\mathrm{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\mathrm{ref}}(y_l|x)}\right)\right] \tag{2.5}$$

where y_w and y_l are the preferred and dispreferred completions, β controls the deviation from the reference policy π_{ref}, and σ is the logistic sigmoid. The gradient implicitly increases the log-likelihood of preferred outputs and decreases that of dispreferred ones, weighted by how much the current policy already agrees. DPO matched or exceeded PPO-based RLHF on sentiment, summarisation, and dialogue tasks while being simpler to implement and more stable to train.

Application to PCG Arena. MarioDPO applies DPO to Mario level generation. Preference pairs are constructed from PCG Arena votes: the level receiving the human vote becomes y_w, the rejected level becomes y_l. The DPO objective fine-tunes a GPT-2-based level generator to produce levels more aligned with human preferences, closing the loop from human evaluation back to generation. The training procedure is detailed in Chapter 5.

Chapter 3

PCG Arena — System Design

This chapter presents the design and implementation of PCG Arena, a web-based platform for blind A/B testing of procedural content generators for Super Mario Bros. We describe the system architecture, game engine, battle and voting mechanics, the novel AGIS matchmaking algorithm, the Glicko-2 rating integration, the builder submission workflow, and the deployment infrastructure.

3.1 Requirements and Design Goals

The primary goal of PCG Arena is to enable rigorous, human-centered evaluation of PCG algorithms through pairwise comparisons. Translating this goal into a concrete system imposes several requirements:

1. **Blind comparison.** Players must not know which generator produced which level, eliminating brand-loyalty bias and ensuring votes reflect genuine quality perception.

2. **Browser-based gameplay.** No installation should be required; the full Super Mario Bros experience must run in the browser with faithful physics.

3. **Uncertainty-aware ranking.** The rating system must quantify confidence in each generator's ranking, not merely produce a point estimate.

4. **Efficient matchmaking.** With a limited player population, every battle should maximise information gain toward stable rankings.

5. **Open submission.** Any researcher should be able to submit their own generator and receive a rating, with minimal friction.

6. **Rich telemetry.** Beyond the binary vote, the platform should capture gameplay trajectories, qualitative tags, and timing data for downstream analysis.

3.2 System Architecture

PCG Arena follows a three-tier web architecture with clear separation of concerns: a *presentation layer* (browser-based frontend), an *application layer* (RESTful backend), and a *data layer* (relational database). Table 3.1 summarises the technology choices for each layer.

Layer	Technology	Rationale
Frontend	React 18 + TypeScript 5.6 (Vite)	Canvas rendering; type safety
Backend	Python 3.12, FastAPI	Auto OpenAPI docs; Pydantic validation
Database	SQLite	Single-file; ACID; zero-config
Proxy	Caddy	Automatic TLS via Let's Encrypt
Container	Docker + docker-compose	Reproducible deployment

Table 3.1: PCG Arena technology stack.

3.2.1 Frontend

The frontend is a single-page application built with React 18 and TypeScript 5.6, bundled by Vite. Its source tree is organised into five top-level directories:

```
frontend/src/
  api/ # Backend communication (fetch wrappers)
  engine/ # Mario game engine (ported from Java)
  components/ # Reusable React UI components
  pages/ # Full-page route components
  contexts/ # React context providers (auth, theme)
```

Game rendering uses the HTML5 Canvas API for direct pixel manipulation, while the rest of the UI is standard React. Vite provides sub-second hot-module replacement during development and produces an optimised static bundle for production.

3.2.2 Backend

The backend is implemented in Python using FastAPI, chosen for its automatic OpenAPI documentation generation, native `async/await` support, and built-in request validation through Pydantic models. Key modules include:

- `main.py` — route definitions and middleware (CORS, rate limiting)
- `auth.py` — Google OAuth 2.0 and email/password authentication
- `glicko2.py` — Glicko-2 rating algorithm
- `matchmaking.py` — AGIS matchmaking algorithm
- `builders.py` — generator submission and management

- `stats.py` — analytics and data export endpoints

The API is versioned under the `/v1/` prefix and exposes endpoints for battles, votes, statistics, authentication, builder operations, and admin management. Administrative endpoints additionally require either session-based admin access or an API key.

3.2.3 Database

SQLite was selected as the database engine due to its single-file storage (simplifying backups and deployment), ACID transaction guarantees, and zero-configuration operation. The schema is managed through 17 sequential SQL migrations and comprises the following tables:

Table	Purpose
`generators`	PCG algorithm metadata and ownership
`levels`	ASCII tilemap content with dimensions and content hash
`battles`	Pairwise comparison instances (left/right level pairing)
`votes`	User preference outcomes with telemetry and tags
`ratings`	Current Glicko-2 state (μ, RD, σ) per generator
`rating_events`	Audit log of all rating changes
`generator_pair_stats`	Confusion matrix data (pairwise win/loss counts)
`users`	Authenticated user accounts
`user_sessions`	Active login sessions
`email_verifications`	Email verification tokens
`password_resets`	Password reset tokens
`player_profiles`	Anonymous player identities
`player_sessions`	Player session tracking
`level_stats`	Aggregate per-level gameplay statistics
`play_trajectories`	Detailed player position data for heatmaps
`level_features`	Structural feature extraction (tiles, enemies, gaps)
`schema_migrations`	Migration bookkeeping

Table 3.2: Database schema: all 17 tables.

3.3 Game Engine

A critical component of PCG Arena is the browser-based Super Mario Bros engine, which was ported from the Java-based Mario AI Framework [9] to TypeScript. The port required faithful preservation of gameplay physics while adapting rendering and input handling to browser constraints.

3.3.1 TypeScript Port

The porting process involved four main translation steps:

1. **Class structure.** Java classes were converted to TypeScript classes, preserving inheritance hierarchies (e.g. `MarioSprite` → `Mario`, `Enemy`).
2. **Physics constants.** All numerical constants were kept identical to the Java originals (Table 3.3).
3. **Rendering.** Java Swing/AWT rendering was replaced with the HTML5 Canvas API (`CanvasRenderingContext2D`).
4. **Input.** Keyboard event handling was reimplemented using browser `KeyboardEvent` listeners.

The key engine modules are:

```
engine/
  MarioGame.ts # Game loop (requestAnimationFrame)
  MarioWorld.ts # World state, collision, game logic
  MarioLevel.ts # ASCII tilemap parser
  MarioSprite.ts # Base sprite class (physics, rendering)
  sprites/ # Mario, enemies, items, projectiles
  graphics/ # Camera, sprite sheets
  effects/ # Particle effects
```

Parameter	Value
Ground inertia	0.89
Air inertia	0.89
Gravity	3.0
Jump velocity	−1.9
Walk speed	0.6
Run speed	1.2

Table 3.3: Core physics constants preserved from the Java Mario AI Framework.

The game loop runs at 30 ticks per second, matching the original Java framework. Rendering runs separately at 60 FPS via `requestAnimationFrame`, interpolating between physics frames. This decoupling preserves the exact physics behaviour of the original framework (which assumes 30-tick updates) while providing visually smooth animation in the browser. Physics updates are fully deterministic, ensuring consistent gameplay across sessions.

3.3.2 Level Format

Levels are stored as plain-text ASCII tilemaps, where each character encodes a tile type or entity spawn point. The format supports 37 distinct tile characters, summarised in Table 3.4.

Char	Meaning	Char	Meaning
`-`	Air (empty)	`Q`	Question block (coin)
`X`	Solid floor	`!`	Question block (coin, alt)
`S`	Brick block	`C`	Coin block
`#`	Pyramid block	`U`	Mushroom block
`D`	Used block	`L`	1-Up block
`%`	Jump-through platform	`1`	Invisible 1-Up block
`|`	Platform background	`2`	Invisible coin block
`?`	Question block (mushroom)	`o`	Free-standing coin
`@`	Question block (mushroom, alt)	`M`	Mario spawn
`E`	Goomba	`F`	Flag (level exit)
`g`	Goomba (alt)	`t`	Pipe (empty)
`G`	Winged Goomba	`T`	Pipe (flower)
`k`	Green Koopa	`<`	Pipe top left
`K`	Winged Green Koopa	`>`	Pipe top right
`r`	Red Koopa	`[`	Pipe body left
`R`	Winged Red Koopa	`]`	Pipe body right
`y`	Spiky	`*`	Bullet Bill body
`Y`	Winged Spiky	`B`	Bullet Bill head
		`b`	Bullet Bill neck

Table 3.4: Complete ASCII tile legend (37 characters).

A typical level file consists of 16 rows (the default height) and up to several hundred columns, stored with newline-separated rows. For example, the original Super Mario Bros World 1-1 is encoded as a 202×16 character grid. Figure 3.1 shows the segment of this level:

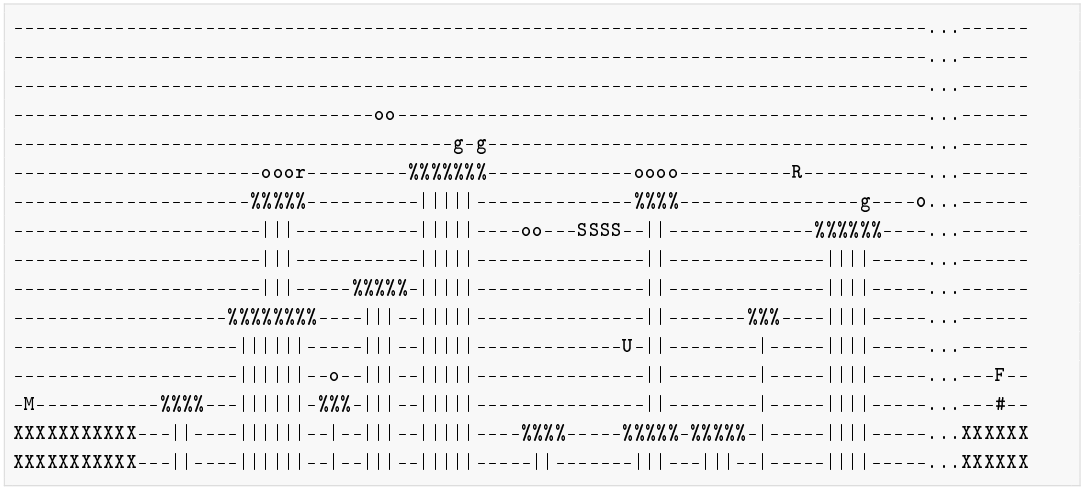

Figure 3.1: Original Super Mario Bros Level in ASCII tilemap format. `M` marks Mario's spawn; `X` is solid ground; `U` and `S` are question blocks; `g` is a Goomba; `F` denotes level exit; `-` is empty air.

3.3.3 Level Validation

Every submitted level undergoes format validation before insertion into the database. The validation procedure, implemented in the backend's `seed.py` module, checks:

1. **Height bounds:** the number of rows must satisfy $10 \leq h \leq 20$.
2. **Width bounds:** the number of columns must satisfy $150 \leq w \leq 250$. In practice, all submitted generators produce levels with widths between 150 and 208 columns; the lower bound of 1 is deliberately permissive and may be tightened in a future revision.
3. **Rectangular shape:** all rows must have identical length.
4. **Character whitelist:** every character must belong to the set of 37 allowed tiles (Table 3.4).

Levels that fail any check are rejected with a descriptive error message. Note that the platform performs *format-level* validation only; there is no automated playability or reachability analysis (e.g. checking that the exit is reachable from the spawn). Content hashes are computed for deduplication.

3.4 Battle and Voting System

The core user experience is the *battle*: a blind A/B test in which the player compares two levels produced by different generators, not knowing which generator produced which level.

3.4.1 Battle Flow

A battle proceeds through five stages:

1. **Battle request.** The frontend calls `POST /v1/battles:next`.

2. **Matchmaking.** The AGIS algorithm (Section 3.5) selects two generators; one random level is drawn from each.

3. **Gameplay.** The player plays both levels sequentially (labelled "Left" and "Right"). During play, the engine records a trajectory of Mario's position and death events.

4. **Voting.** After completing both levels, the player chooses one of four options: *Left is Better*, *Right is Better*, *Tie*, or *Skip*. The "Skip" option, used when the player feels unable to judge or encounters a technical issue, does not affect ratings.

5. **Rating update.** The backend updates Glicko-2 ratings for both generators atomically within a database transaction (Section 3.6).

Figure 3.2 shows the main gameplay interface. After completing both levels, the voting panel (Figure 3.3) allows the player to cast their preference and optionally tag each level.

Figure 3.2: Main gameplay view: the player experiences a procedurally generated level in the browser-based Mario engine.

Figure 3.3: Voting panel shown after both levels are played, with preference buttons and optional tag selection.

3.4.2 Tagging System

In addition to the preference vote, players may optionally tag each level in the battle with up to three descriptive labels drawn from a fixed vocabulary of seven tags:

Tag	Description
`fun`	The level was enjoyable to play
`boring`	The level lacked engagement
`creative`	The level showed novel or interesting design
`too_hard`	The level was excessively difficult
`too_easy`	The level was trivially easy
`impossible`	The level appeared to be uncompletable
`broken_graphics`	The level had visual or rendering issues

Table 3.5: The seven available tags for level characterisation.

Tags are stored as boolean columns in the `votes` table and aggregated in `level_stats`, enabling analyses such as correlating perceived difficulty with win rates or identifying generators whose levels are consistently tagged as `impossible`.

3.5 Matchmaking: AGIS Algorithm

3.5.1 Problem Definition

Given n generators with current Glicko-2 ratings and uncertainties, the matchmaking problem is to select a pair for the next battle that maximises information gain toward accurate final rankings. Naive strategies are suboptimal:

- **Uniform random** wastes battles on already-converged pairs.
- **Most-uncertain-first** neglects head-to-head comparisons between similarly rated generators.
- **Round-robin** ignores rating information and pairs dissimilar generators.

To address these shortcomings, we designed **AGIS (Adaptive Glicko-Informed Selection)**, a custom two-stage matchmaking algorithm that incorporates rating uncertainty, skill similarity, and pairwise coverage into generator selection. While the individual components draw on well-known ideas from information-theoretic experimental design (uncertainty sampling, coverage balancing), their combination into a lightweight, Glicko-2-aware matchmaker tailored to the PCG evaluation setting is, to our knowledge, novel.

3.5.2 Stage 1: First Generator Selection

Each generator i receives a selection weight w_i composing an uncertainty term and a games-played term:

$$w_i = \left(1 + \widetilde{\mathrm{RD}}_i\right)^2 \cdot \mathrm{boost}(\mathrm{games}_i) \tag{3.1}$$

where $\widetilde{\mathrm{RD}}_i = (\mathrm{RD}_i - \mathrm{RD}_{\min})/(\mathrm{RD}_{\max} - \mathrm{RD}_{\min})$ is the normalised rating deviation. The boost function provides a $4\times$ weight multiplier for generators with zero games, linearly decaying to $1\times$ at `MIN_GAMES` (default: 30). After convergence, a mild quality bias is applied:

$$\mathrm{boost}_{\mathrm{converged}} = 0.8 + q_{\mathrm{bias}} \cdot \min\Big(1,\ \max\big(0,\ \tfrac{\mu_i - 600}{800}\big)\Big) \tag{3.2}$$

where $q_{\mathrm{bias}} = 0.2$ is the configurable quality bias strength. The weights are normalised to sum to 1, forming a categorical (discrete) probability distribution; the first generator is then drawn from this distribution via weighted random sampling.

3.5.3 Stage 2: Second Generator Selection

The intuition behind Stage 2 is that the most informative battle is one between generators of *similar* strength (maximising discriminative power) whose ratings are

still *uncertain* (maximising information gain per battle), while ensuring that every pair accumulates enough head-to-head data for a reliable confusion matrix. The four weighted components below formalise these intuitions.

Given the first generator g_1, the second is selected using a composite pair weight:

$$w_{1,j} = \alpha \cdot S(g_1, g_j) \; + \; \beta \cdot U(g_j) \; + \; \gamma \cdot I(g_1, g_j) \; + \; \mathrm{cov}(g_1, g_j) \tag{3.3}$$

The four components are:

- **Rating similarity** $S(g_1, g_j) = \exp\!\left(-\Delta\mu^2 / 2\sigma_s^2\right)$, a Gaussian kernel with $\sigma_s = 150$ penalising large rating gaps.
- **Uncertainty** $U(g_j) = 1 + \widetilde{\mathrm{RD}}_j$, favouring uncertain generators (range $[1, 2]$).
- **Information gain** $I(g_1, g_j)$, incorporating the joint RD and a match-quality estimate.
- **Coverage bonus** $\mathrm{cov}(g_1, g_j)$: for pairs with fewer than `TARGET_BATTLES_PER_PAIR` (default: 10) battles, an exponential bonus $2 \cdot e^{-c/3}$ is applied, where c is the current battle count; otherwise a residual 0.1 is used.

3.5.4 Parameters

Table 3.6 lists the configurable AGIS parameters and their defaults, which are set via environment variables.

Parameter	Description	Default
α	Rating similarity weight	0.5
β	Uncertainty weight	0.3
γ	Information gain weight	0.2
`MIN_GAMES`	Games before "converged"	30
`TARGET_BATTLES_PER_PAIR`	Min. battles per pair	10
`RATING_SIGMA`	Similarity kernel σ_s	150.0
`QUALITY_BIAS`	Quality bias strength q_{bias}	0.2

Table 3.6: AGIS algorithm parameters and default values.

3.6 Glicko-2 Rating System

PCG Arena uses the Glicko-2 rating system [7] to maintain uncertainty-aware rankings. Each generator carries three parameters:

- **Rating (μ):** the skill estimate, displayed on a scale centred at 1000.
- **Rating Deviation (RD, ϕ):** the uncertainty in the rating. High RD indicates an unreliable estimate; low RD indicates a well-established rating.
- **Volatility (σ):** the expected fluctuation in performance over time.

3.6.1 Expected Score

The expected score when generator A faces generator B is:

$$E(\mu_A, \mu_B, \phi_B) = \frac{1}{1 + \exp\big(-g(\phi_B)(\mu_A - \mu_B)\big)} \tag{3.4}$$

where $g(\phi) = 1/\sqrt{1 + 3\phi^2/\pi^2}$ down-weights the prediction when the opponent's rating is uncertain.

3.6.2 Rating Update Procedure

After each battle, the winning generator receives a score of 1, the loser 0, and ties yield 0.5 for both. The update proceeds as follows:

1. Convert ratings to the internal Glicko-2 scale (centred at 0, divided by $\lambda = 173.7178$). This constant equals $400/\ln 10 \approx 173.7178$ and bridges the Elo convention (400 points per tenfold win-probability change) with the natural logistic function used internally by Glicko-2.
2. Compute the variance v from the opponent's RD.
3. Compute $\Delta = v \cdot g(\phi_B)(s - E)$, where s is the actual score.
4. Update volatility σ' via the iterative Illinois algorithm (system constant $\tau = 0.5$).
5. Update RD: $\phi' = 1/\sqrt{1/\phi^{*2} + 1/v}$, where $\phi^* = \sqrt{\phi^2 + \sigma'^2}$.
6. Update rating: $\mu' = \mu + \phi'^2 \cdot g(\phi_B)(s - E)$.
7. Convert back to the display scale and clamp to $[100, 3000]$.

All rating updates are performed atomically within a single database transaction, with an audit record written to the `rating_events` table.

3.6.3 Configuration

Table 3.7 lists the Glicko-2 constants used by PCG Arena.

Parameter	Value
Initial rating (μ_0)	1000
Initial RD (ϕ_0)	350
Initial volatility (σ_0)	0.06
System constant (τ)	0.5
Scale factor (λ)	173.7178
Minimum RD	30
Maximum RD	350
Minimum rating	100
Maximum rating	3000

Table 3.7: Glicko-2 configuration parameters.

The RD of each generator provides a natural 95% confidence interval for its true rating: $\mathrm{CI}_{95\%} = [\mu - 1.96\,\mathrm{RD},\ \mu + 1.96\,\mathrm{RD}]$. However, the public leaderboard displays only the point-estimate rating; confidence intervals are not shown on the platform's frontend. Figure 3.4 shows the leaderboard as seen by players.

Figure 3.4: Public leaderboard ranking generators by their Glicko-2 rating.

Clicking a generator reveals detailed statistics including rating history, per-opponent win rates, a level gallery, and tag distributions (Figure 3.5).

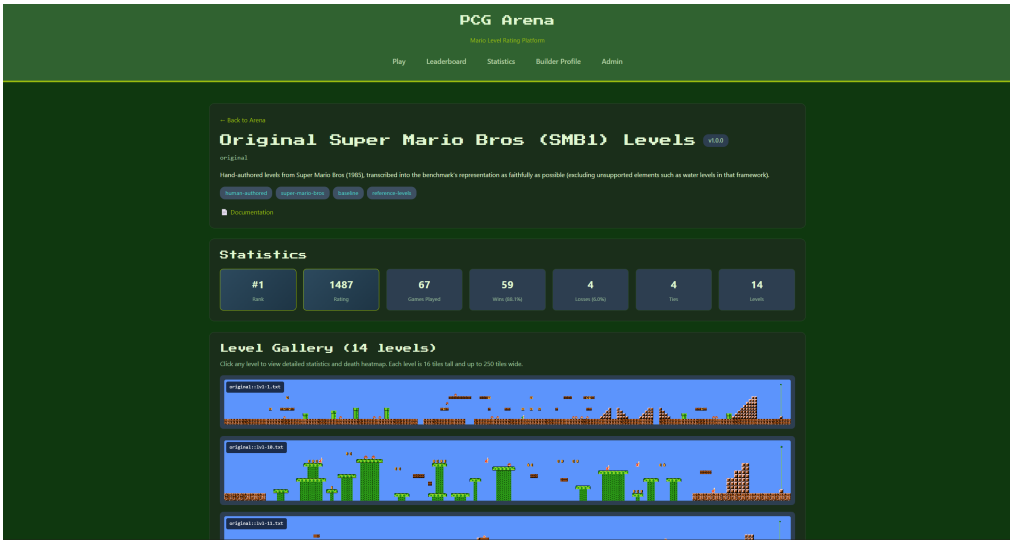

Figure 3.5: Generator detail page showing level gallery and performance statistics.

3.7 Builder Profile and Generator Submission

Authenticated users may submit their own generators through the Builder Profile page. The submission workflow is:

1. The user provides generator metadata: name, version, description, and a URL to the generator's source repository.
2. The user uploads a ZIP archive containing level files in the ASCII tilemap format.
3. The backend validates every level (Section 3.3.3). Levels that fail validation are rejected with detailed error messages.
4. Valid levels and the generator record are inserted into the database.
5. An initial Glicko-2 rating of 1000 ± 350 is assigned.
6. The generator appears on the leaderboard immediately and enters the matchmaking pool.

Each user may own at most three generators. When a user wishes to remove a generator, the system distinguishes two cases. If the generator has already participated in battles, it is *soft-deleted*: marked as inactive and excluded from future matchmaking, but its records (levels, ratings, votes) are retained so that historical analyses and the confusion matrix remain consistent. If the generator has never appeared in a battle, it is *hard-deleted* (fully removed from the database) since no other data depends on it.

Figure 3.6 shows the Builder Profile page.

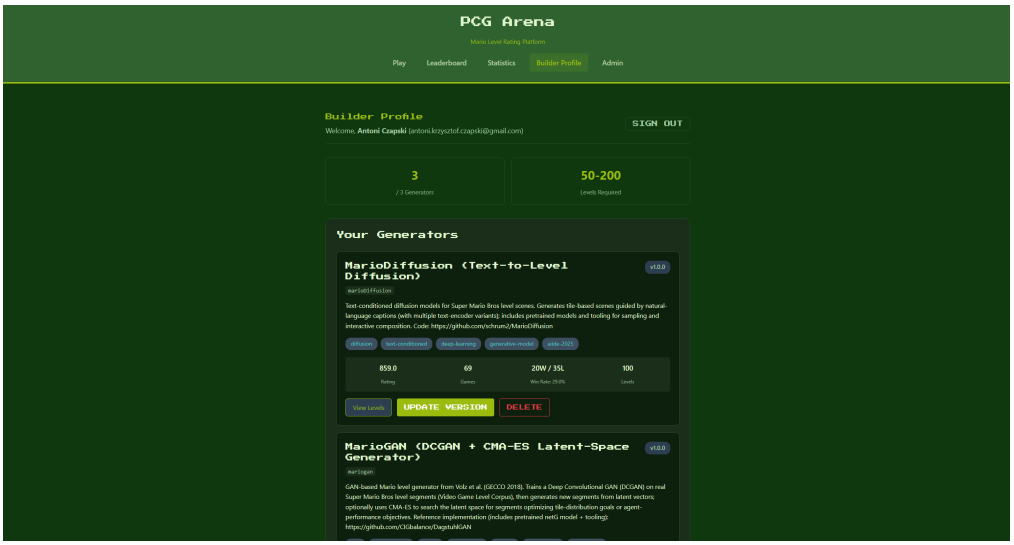

Figure 3.6: Builder Profile page for submitting and managing generators.

3.8 Authentication

PCG Arena supports two authentication methods:

- **Google OAuth 2.0.** The frontend opens a Google sign-in popup; the returned ID token (JWT) is sent to `POST /v1/auth/google`, where the backend verifies it against Google's servers. Google users are automatically marked as email-verified.
- **Email and password.** Users register with an email, password, and display name. Passwords are hashed with bcrypt (work factor 12). A verification email is sent via Twilio SendGrid containing a unique token (64 hex characters, 24-hour expiry).

Upon successful authentication, the backend issues a session cookie (`arena_session`) containing a 256-bit cryptographically random token. Sessions expire after seven days. HTTP-only cookies prevent client-side script access.

Administrative access is granted to users whose email addresses appear in the `ARENA_ADMIN_EMAILS` configuration list.

3.9 Deployment and Infrastructure

PCG Arena is deployed on Google Cloud Platform using a single Compute Engine instance (`e2-micro`, Ubuntu 22.04 LTS). The backend runs inside a Docker container

orchestrated by `docker-compose`, with the SQLite database file bind-mounted from the host for persistence across container rebuilds.

Caddy serves as the reverse proxy, handling:

- Automatic TLS certificate provisioning and renewal (Let's Encrypt).
- HTTP-to-HTTPS redirection.
- Reverse proxying API requests to `localhost:8080`.
- Static file serving for the frontend production build.

The platform is accessible at `https://pcg-arena.com`. The backend binds to `127.0.0.1:8080` and is not exposed to the public internet directly; all external traffic passes through Caddy. Firewall rules allow only ports 80 and 443. Database backups are performed via a scheduled script that copies the SQLite file to a timestamped archive.

Chapter 4

User Study and Exploratory Data Analysis

4.1 Study Design and Data Collection

tbd

4.2 Dataset Overview

tbd

4.3 Generator Rankings and Performance

tbd

4.4 RQ1: What Makes a Good Level?

4.4.1 Difficulty and the Flow Channel Hypothesis

tbd

4.4.2 Structural Variety and Creativity

tbd

4.4.3 Path Freedom and Player Agency

tbd

4.4.4 Hazard Difficulty and Leniency

tbd

4.5 RQ2: Player Preference Consistency

4.5.1 Preference Clusters

tbd

4.5.2 Skill–Consistency Relationship

tbd

4.6 RQ3: Tag Semantics and Validation

4.6.1 Tag–Telemetry Correspondence

tbd

4.6.2 Feature Importance for Tags

tbd

4.6.3 Predictive Modelling of Preferences

tbd

4.7 Trajectory Analysis

tbd

4.8 Summary of Findings

tbd

Chapter 5

MarioDPO — Preference-Aligned Level Generation

5.1 Motivation and Approach

tbd

5.2 Judge Function Development

tbd

5.2.1 Verticality Reward (Experiment A)

tbd

5.2.2 Hazard Hierarchy (Experiment B)

tbd

5.2.3 Style Matching (Experiment D)

tbd

5.2.4 Final Judge Function

tbd

5.3 Training Pipeline

5.3.1 Supervised Fine-Tuning (SFT)

tbd

5.3.2 Preference Dataset Construction

tbd

5.3.3 DPO Training

tbd

5.4 Column-Major Tokenisation

tbd

5.5 Results

tbd

5.6 Analysis and Discussion

tbd

Chapter 6

Conclusions and Future Work

6.1 Summary of Contributions

tbd

6.2 Limitations

tbd

6.3 Future Directions

tbd

Bibliography

[1] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. doi: 10.1093/biomet/39.3-4.324.

[2] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. Chatbot arena: An open platform for evaluating LLMs by human preference. *arXiv preprint arXiv:2403.04132*, 2024. URL `https://arxiv.org/abs/2403.04132`.

[3] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL `https://arxiv.org/abs/1706.03741`. arXiv:1706.03741.

[4] Steve Dahlskog and Julian Togelius. Procedural content generation using patterns as objectives. In *Applications of Evolutionary Computation (EvoApplications)*, volume 8602 of *Lecture Notes in Computer Science*, pages 325–336. Springer, 2014. doi: 10.1007/978-3-662-45523-4_27.

[5] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco, 1978.

[6] Mark E. Glickman. Parameter estimation in large dynamic paired comparison experiments. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 48(3):377–394, 1999. doi: 10.1111/1467-9876.00120.

[7] Mark E. Glickman. Example of the Glicko-2 system. Technical report, Boston University, 2012. URL `http://www.glicko.net/glicko/glicko2.pdf`.

[8] Britton Horn, Steve Dahlskog, Noor Shaker, Gillian Smith, and Julian Togelius. A comparative evaluation of procedural level generators in the Mario AI framework. In *Proceedings of the 9th International Conference on the Foundations of Digital Games (FDG)*, 2014. URL `https://www.fdg2014.org/papers/fdg2014_paper_14.pdf`.

[9] Sergey Karakovskiy and Julian Togelius. The Mario AI benchmark and competitions. *IEEE Transactions on Computational Intelligence and AI in Games*, 4 (1):55–67, 2012. doi: 10.1109/TCIAIG.2012.2188528.

[10] Ahmed Khalifa, Philip Bontrager, Sam Earle, and Julian Togelius. PCGRL: Procedural content generation via reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2020. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/7416`. arXiv:2001.09212.

[11] José Rafael Hernández Mariño, Walber Moreira Penna Reis, and Levi H. S. Lelis. An empirical evaluation of evaluation metrics of procedurally generated Mario levels. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2015. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/12785`.

[12] Peter Mawhorter and Michael Mateas. Procedural level generation using occupancy-regulated extension. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, pages 351–358. IEEE, 2010. doi: 10.1109/ITW.2010.5593333.

[13] Seong-Geon Nam, Chih-Hung Hsueh, Pasit Rerkjirattikal, and Kokolo Ikeda. Using RL to generate levels of Super Mario Bros with quality and diversity. *IEEE Transactions on Games*, 16(4):807–820, 2024.

[14] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL `https://arxiv.org/abs/2203.02155`. arXiv:2203.02155.

[15] Christopher Pedersen, Julian Togelius, and Georgios N. Yannakakis. Modeling player experience for content creation. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(1):54–67, 2010.

[16] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL `https://arxiv.org/abs/2305.18290`. arXiv:2305.18290.

[17] Anurag Sarkar and Seth Cooper. Level difficulty and player skill prediction in human computation games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2017. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/12948`.

[18] Jacob Schrum, Olivia Kilday, Emilio Salas, Bess Hagan, and Reid Williams. Text-to-level diffusion models with various text encoders for Super Mario Bros. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive*

Digital Entertainment (AIIDE), pages 110–120, 2025. doi: 10.1609/aiide.v21i1.36815. URL https://ojs.aaai.org/index.php/AIIDE/article/view/36815.

[19] Noor Shaker, Julian Togelius, Georgios N. Yannakakis, Ben Weber, Tomoyuki Shimizu, Tomonori Hashiyama, Nathan Sorenson, Philippe Pasquier, Peter Mawhorter, Glen Takahashi, Gillian Smith, and Robin Baumgarten. The 2010 Mario AI championship: Level generation track. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(4):332–347, 2011. doi: 10.1109/TCIAIG.2011.2166267.

[20] Noor Shaker, Georgios N. Yannakakis, and Julian Togelius. Feature analysis for modeling game content quality. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, 2011. doi: 10.1109/CIG.2011.6031998.

[21] Noor Shaker, Miguel Nicolau, Georgios N. Yannakakis, Julian Togelius, and Michael O'Neill. Evolving levels for Super Mario Bros using grammatical evolution. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, pages 304–311, 2012. doi: 10.1109/CIG.2012.6374170.

[22] Gillian Smith and Jim Whitehead. Analyzing the expressive range of a level generator. In *Proceedings of the 2010 Workshop on Procedural Content Generation in Games (PCGames)*, pages 4:1–4:7. ACM, 2010. doi: 10.1145/1814256.1814260.

[23] Rion Snow, Brendan O'Connor, Daniel Jurafsky, and Andrew Y. Ng. Cheap and fast — but is it good? Evaluating non-expert annotations for natural language tasks. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 254–263. Association for Computational Linguistics, 2008. URL https://aclanthology.org/D08-1027/.

[24] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2009.01325. arXiv:2009.01325.

[25] Shyam Sudhakaran, Miguel González-Duque, Claire Glanois, Matthias Freiberger, Elias Najarro, and Sebastian Risi. MarioGPT: Open-ended text2level generation through large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/file/a9bbeb2858dfbdbd4c19814e5d80ec60-Paper-Conference.pdf. arXiv:2302.05981.

[26] Adam Summerville and Michael Mateas. Super Mario as a string: Platformer level generation via LSTMs. In *Proceedings of the 1st International*

Joint Conference of DiGRA and FDG, 2016. doi: 10.26503/dl.v2016i1.752. arXiv:1603.00930.

[27] Adam Summerville, José Rafael Hernández Mariño, Sam Snodgrass, Santiago Ontañón, and Levi H. S. Lelis. Understanding Mario: An evaluation of design metrics for platformers. In *Proceedings of the 12th International Conference on the Foundations of Digital Games (FDG)*, 2017. doi: 10.1145/3102071.3102080.

[28] Adam Summerville, Sam Snodgrass, Matthew Guzdial, Christoffer Holmgård, Amy K. Hoover, Aaron Isaksen, Andy Nealen, and Julian Togelius. Procedural content generation via machine learning (PCGML). *IEEE Transactions on Games*, 10(3):257–270, 2018. doi: 10.1109/TG.2018.2846639.

[29] Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186, 2011. doi: 10.1109/TCIAIG.2011.2148116.

[30] Vanessa Volz, Jacob Schrum, Jialin Liu, Simon M. Lucas, Adam Smith, and Sebastian Risi. Evolving Mario levels in the latent space of a deep convolutional generative adversarial network. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, pages 221–228. ACM, 2018. doi: 10.1145/3205455.3205517.

[31] Georgios N. Yannakakis and Julian Togelius. Experience-driven procedural content generation. *IEEE Transactions on Affective Computing*, 2(3):147–161, 2011. doi: 10.1109/T-AFFC.2011.6.

[32] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL `https://arxiv.org/abs/2306.05685`. arXiv:2306.05685.

[33] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019. URL `https://arxiv.org/abs/1909.08593`.